

STANDARD OPERATING PROCEDURE

PROJECT 01 — CUSTOMER CHURN INTELLIGENCE & PREDICTION PLATFORM

1. Project Objective

Build a production-style machine-learning system that predicts whether a customer is likely to leave a product or subscription and provides churn probability, risk level, and interpretable reasons behind the prediction.

2. Problem Statement

Organizations lose revenue when customers leave unexpectedly. The system should identify high-risk customers early so a business can prioritize retention actions.

3. Expected System Flow

Customer Data → Data Validation → Preprocessing → Feature Engineering → Class-Imbalance Handling → Model Training → Evaluation → Churn Probability → Explainability → FastAPI → Web Dashboard

4. Technology Stack

- Python, Pandas, NumPy and Scikit-learn.
- Random Forest and XGBoost/Gradient Boosting, with Logistic Regression as a baseline.
- SHAP for model explainability.
- FastAPI for REST API serving.
- Streamlit or a modern frontend for the dashboard.
- Joblib for model/pipeline persistence.
- Git and GitHub for version control.
- .env for configuration and secrets.

5. Dataset Requirements

- Use the IBM Telco Customer Churn dataset or an equivalent public dataset.
- Validate columns, data types, missing values, duplicates and invalid categories.
- Perform meaningful feature engineering with business interpretation.
- Encode categorical variables correctly.
- Create train/validation/test splits without data leakage.

6. Machine Learning Pipeline

- Create a simple baseline before optimization.
- Train Logistic Regression, Random Forest and XGBoost/Gradient Boosting.
- Handle class imbalance using class weights or appropriate resampling.
- Compare models using Precision, Recall, F1-score, ROC-AUC and confusion matrix.
- Use PR-AUC when appropriate for the imbalance level.
- Select the final model based on the actual business objective.
- Save the complete preprocessing + model pipeline.

7. Explainability

- Use SHAP or an appropriate explainability method.
- Show the strongest factors influencing an individual prediction.
- Clearly present explanations as model-based signals, not causal claims.
- Never fabricate explanations or model results.

8. FastAPI Requirements

Endpoint	Purpose
POST /predict	Return churn probability, risk level and explanation.
GET /health	Return service health status.
GET /model-info	Return model/version information.

9. UI/UX Requirements

- Use an Antigravity-inspired modern AI product aesthetic.
- Clean typography, premium cards, subtle gradients and refined dark/light surfaces.
- Responsive desktop and mobile design.
- Dashboard: total customers, predicted churn, high-risk customers and average churn probability.
- Prediction page: customer form, churn probability, risk level and explanation.
- Risk analysis: searchable/filterable customer table and risk ranking.
- Model performance: real evaluation charts and metrics.
- Implement loading, empty, validation and error states.
- Every button and navigation element must work; no fake UI.

10. Security & Reliability

- Never hard-code API keys or secrets.
- Validate API inputs and configure CORS intentionally.
- Use structured logging and clean error responses.
- Do not expose stack traces or credentials to the frontend.

11. Testing

- Test data preprocessing and feature engineering.
- Test model loading and prediction.
- Test FastAPI endpoints with valid and invalid inputs.
- Test frontend-to-API prediction flow.
- Verify responsive layout and all navigation/actions.

12. Definition of Done

- Real data trains a real model.
- All metrics come from actual evaluation.
- UI → API → model → prediction → explanation works end-to-end.
- README, setup instructions, .env.example and API documentation are included.
- No fake metrics, placeholder charts, dead buttons or unused modules remain.

- Project is clean, reproducible and deployment-ready.

Final Portfolio Standard: The project must look and behave like a real ML product, not a tutorial notebook with a decorative dashboard.